

Question 1 - Heuristic design and admissibility:

Steps –

stepsHeuristic(word):

 for each letter in word:

 if (letter not equal to goal letter):

 increase heuristic cost by 1

 return heuristic cost

The remaining cost it lower-bounds is the cost of changing a letter to a letter that isn't the corresponding letter in the goal word that might be needed to go through. This is because we can't know beforehand how many valid words must be gone through in order to get a goal word letter in the right spot. It cannot overestimate the true optimal remaining cost since any mismatched letters in the current word must be changed one at a time, so there must be at least that number of steps.

Scrabble –

scrabbleHeuristic(word):

 for each letter in word:

 if (letter not equal to goal letter):

 increase heuristic cost by scrabble letter cost of goal letter

 return heuristic cost

The remaining cost it lower-bounds is the same as for the steps cost, is the cost of changing a letter to a letter that isn't the corresponding letter in the goal word that might be needed to go through, for the same reason. It cannot overestimate the true optimal remaining cost since any mismatched letters in the word must be changed one at a time, ending up with the cost to change to that letter. If any transitional words must be used, they will always add to the cost.

Frequency –

frequencyHeuristic(word):

 for each letter in word:

 if (letter not equal to goal letter):

 increase mismatched letters

$\text{return } ((\text{mismatched letters} - 1) * (1 + \text{most frequent word value})) + (1 + \text{cost of goal word})$

The remaining cost it lower-bounds is the same reason as the previous two, the cost of changing to any transitional words that might need to be gone through, for the same reason. It cannot overestimate the true optimal remaining cost, as any transitional words must have a cost equal to or higher than the cheapest frequency cost, and the last word must be the goal word, so its cost is added as the last mismatched letter cost instead of the cheapest cost. The amount of mismatched letters must always be one or greater, as if it was zero, the current word would be the goal word, and be caught before any calculations.

Question 2 - Does the heuristic actually help?:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
Why -> Ask	Steps	5	773	5	16	97.9301%
	Scrabble	9	765	9	13	98.3007%
	Frequency	38.8554	808	38.8554	634	21.5347%
Bear -> Duck	Steps	4	691	4	6	99.1317%
	Scrabble	11	2981	11	6	99.7987%
	Frequency	38.2402	1084	38.2402	301	72.2325%

UCS and A* always returned the same optimal cost.

The cost measure that A* produced the largest percentage reduction in expanded node was the scrabble cost, but only by a small amount compared to the steps heuristic, both being ~98-99% reduction. The cost measure where A* was the least informative was the frequency heuristic, only being 21.5% for why -> ask and 72.2% for bear -> duck.

For the steps cost measure, UCS is essentially a random selection, and for the scrabble cost measure, UCS is actually worse than random, as if the path needs a high-cost letter it must go through a bunch of low-cost letters first. Since the heuristic for both stick somewhat close to the actual cost, this allows for some leading towards the best path. Even though the cost of transitional words is worse in scrabble, since the letter weights make them worse than just the count of them, the fact that UCS is worse for scrabble than it is for steps makes the reduction a little more for scrabble. For the frequency cost measure, the transitional word price is increased by so much, being 1 to 17 with a lot more in the higher frequency costs than in scrabble, and each minimum transitional cost is the cheapest possible one, the heuristic be not as close to the actual remaining cost. Since UCS is about the same as it is for scrabble, having to check the cheaper words before going through a higher cost one that may be needed, the reduction won't be as significant.

Question 3 – Reversibility and direction-dependent costs:

For the steps cost measure, the optimal cost will be the same either way, as every step has a cost of one, no matter what is changed.

For the scrabble cost measure, since costs are weighted by the letter changed, since alongside any transitional letters, the letters in W_2 must be changed to when going from W_1 to W_2 , whereas the letters in W_1 must be changed to when going from W_2 to W_1 . Since the cost of all the letters in W_1 and W_2 aren't always the same, it will not always be the same optimal cost for W_2 to W_1 as it is for W_1 to W_2 . As an example, any $\rightarrow$ hug has a path of: any ant art ort out hut hug. Going from W_1 to W_2 , the letters that are changed to are – t, r, o, u, h, g, giving a total cost of 10, while going from W_2 to W_1 , the letters that are changed to are – t, o, r, a, n, y, giving a total cost of 9.

For the frequency cost measure, since the W_1 and W_2 aren't guaranteed to have the same frequency cost, and the initial word doesn't contribute to the frequency cost, it will not always be the same optimal cost for W_2 to W_1 as it is for W_1 to W_2 . For example, dog $\rightarrow$ can has a path of: dog don dan can. Going to don is a cost of 6.285653, going to dan is cost of 6.383021, and going to can is a cost of 2.4666543, giving a total cost of 15.1353283. Going to dan and don are the same, but going to dog is a cost of 6.819885, giving a total cost of 19.488559.

Question 4 – LLM heuristic audit:

Prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Heuristic:

- n is a lowercase word of length 3 or 4.
- n must be in the supplied dictionary.
- g is the goal word.
- $h(n)$ is the estimated remaining cost from state n to the goal g .
- $d(n) = \text{Hamming}(n, g)$ is how many letter positions differ between the current word n and the goal word g .
- $m(n)$ is the minimum rarity among all legal one-letter neighbors of n .

w is a **candidate next word** — any legal word you could move to from n by changing exactly one letter.

- $r(w)$ is the rarity of that candidate word w .

$$h'(n) = \begin{cases} 0, & d(n) = 0, \\ 1 + r(g), & d(n) = 1, \\ d(n) + m(n) + r(g), & d(n) \geq 2. \end{cases}$$

The model used was deepseek-v4.1-flash.

Pseudocode for heuristic:

heurFrequency(word, goal):

if (word == goal):

return 0

if (mismatches(word, goal) == 1):

return 1 + frequencyCost(goal)

else:

return mismatches(word, goal) + minNeighborFrequencyCost(word) + frequencyCost(goal)

This heuristic is admissible since the minimum number of word changes is equal to the number of mismatched letters in the current word, as it goes 1 at a time. When the number of mismatched letters is 1, the exact cost of going to the goal is used, and when it is larger, the neighbor of the current word with the lowest frequency cost is added alongside the number of mismatches and the frequency cost of the goal. Since the neighbor of the current word with the lowest frequency cost has a cost equal to or lower than the actual next word in the path and frequency cost of the goal word will always be added, this will always be equal to or lower than the actual cost of getting to the goal word.

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
Why -> Ask	Steps	5	773	5	16	97.9301%
	Scrabble	9	765	9	13	98.3007%
	Frequency	38.8554	808	38.8554	497	38.4901%
Bear -> Duck	Steps	4	691	4	6	99.1317%
	Scrabble	11	2981	11	6	99.7987%
	Frequency	38.2402	1084	38.2402	201	81.4576%

Since the reduction is greater for the AI heuristic while still finding the optimal cost, the proposed heuristic is valid to use and better than my version.